

Recommender Systems

COMPSCI 753



THE UNIVERSITY OF
AUCKLAND
Te Whare Wānanga o Tāmaki Makaurau
NEW ZEALAND

Overview

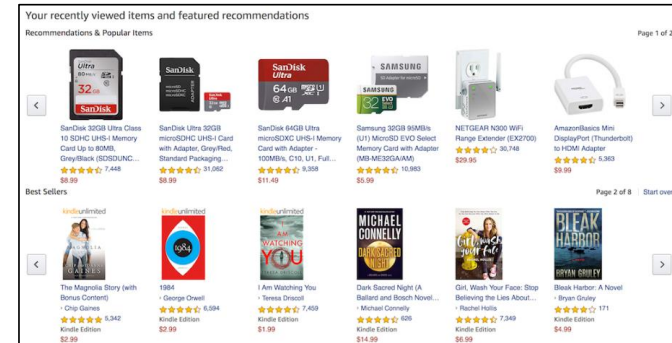
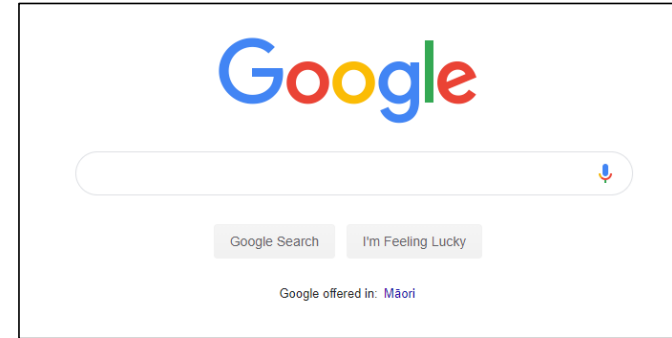
- What is recommender system (RS)?
- Recommendation algorithms
 - Content-based methods
 - Collaborative filtering
 - Latent factor model
- Advanced topics in recommendations (optional)
 - Context-aware recommendations
 - Session-based recommendations

What is recommender system?

- Recommender systems are a way of suggesting like or similar items for users.
- Recommender systems try to automate the process of finding other people with similar tastes and then ask them to suggest new things.
- Recommender systems can change the traditional way that a user obtain information

Search v.s. recommendation

- Search
 - A passive way for information to be retrieved
 - You know what you are looking for
- Recommendation
 - Active way of information delivery
 - You can discover wonderful things that you even don't know them!



Why do we need it?

- Information overload
 - Data gets bigger than ever
 - People are facing more data to consume. How to choose?
- Some statistics
 - According to IBM - **2.5 quintillion bytes** of data are produced every day
 - Statista (March 2019):
 - YouTube users watch **4.5 million** videos every day
 - More than **390K** apps downloaded every day
 - More than **347K** user scrolling on Instagram

The value of recommendations

- Recommender systems can largely increase the revenue of the online business
 - Netflix: 2/3 of the movies watched are recommended
 - Google News: 38% more clickthrough rate from recommendations
 - Amazon: 30% sales from recommendation

[Statistics from Xavier Amatriain]

Industrial recommender systems



A brief trace of recommender systems

- 1992 -- Goldberg proposed **Tapestry**, a personalized mail recommender system based on collaborative filtering
- 1994 -- Resnick proposed **Grouplens**, a collaborative filtering system for news.
- 1996 -- The concept of “recommender system” was established
- 2006 -- **Netflix Prize**, a 1 million competition on recommendation algorithms
- 2007 -- The first ACM conference of recommender system (**RecSys**)
- 2016 -- Google proposed the **Wide & deep** system for scaling the recommender system to big data setting

The recommendation problem

- Predict how a user likes an item based on
 - Similar items to the past consumed items
 - Items that are consumed by others with similar preference
 - Context, i.e., session, cart
- A simple setting - rating prediction

	I1	I2	I3	I4
U1	5	?	?	3
U2	?	4	?	?
U3	3	?	2	?
U4	?	3	?	2
U5	?	5	?	3

Explicit feedback

Ratings, Liked, Favorite

Implicit feedback

Click, Browse, Stay Time

Challenges in RS

- Key problems

- *Sparsity*:

- The user-item interaction matrix is often extremely sparse
 - Millions of items, but a user can only interact with small number of items

- *Cold-start*:

- New items have no ratings yet
 - New users have no historical behavior

Item cold

	I1	I2	I3	I4	I5
U1	5	?	?	3	?
U2	?	4	?	?	?
U3	3	?	2	?	?
U4	?	3	?	2	?
U5	?	5	?	3	?
U6	?	?	?	?	?

User cold

Recommendation algorithms

- Content-based
 - Recommend based on the attributes of items, e.g., functionality, topics, types, etc.
- Collaborative Filtering (CF)
 - User-based: recommend items that similar users liked
 - Item-based: recommend similar items to the items the user liked

Content-based recommendation

- Content-based methods look for the information of items, e.g., description, attributes, etc.
- **Idea**: recommend similar items to those have been highly rated by the user, for instance:
 - Movies with the same actors
 - News with the same topic
 - Items within the same category
- May need to do feature engineering / machine learning to model the content information

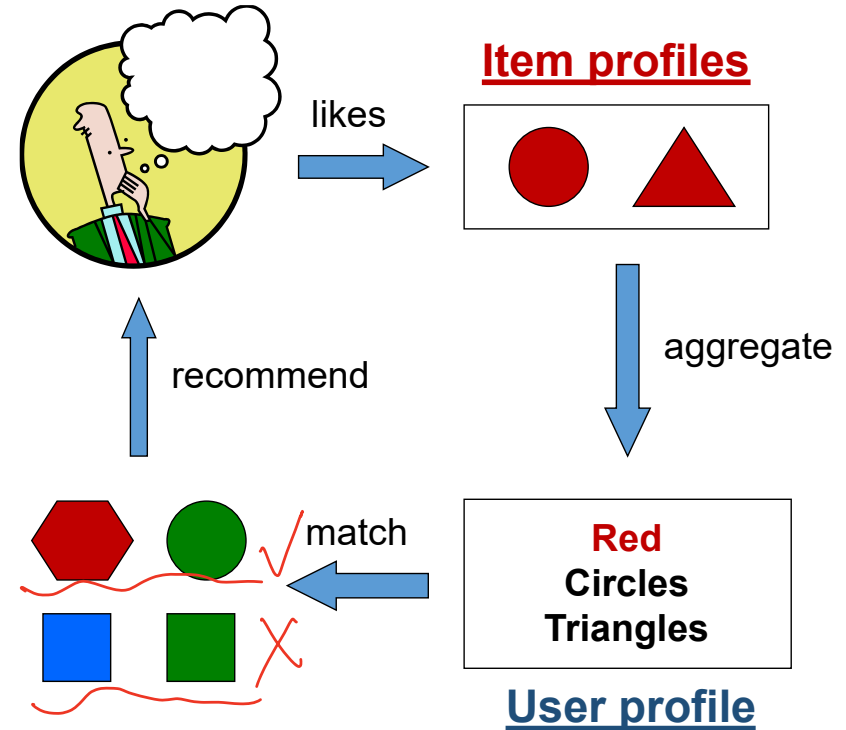
Content of items

- The content of items varies
 - Textual content
 - Amazon – reviews, description of items
 - Twitter – message
 - Attributes
 - Amazon – for computer, we have brand, type, screen, etc.
 - Netflix – year, genre, actors
 - Multimedia
 - Instagram – images, videos

Content-based recommendation

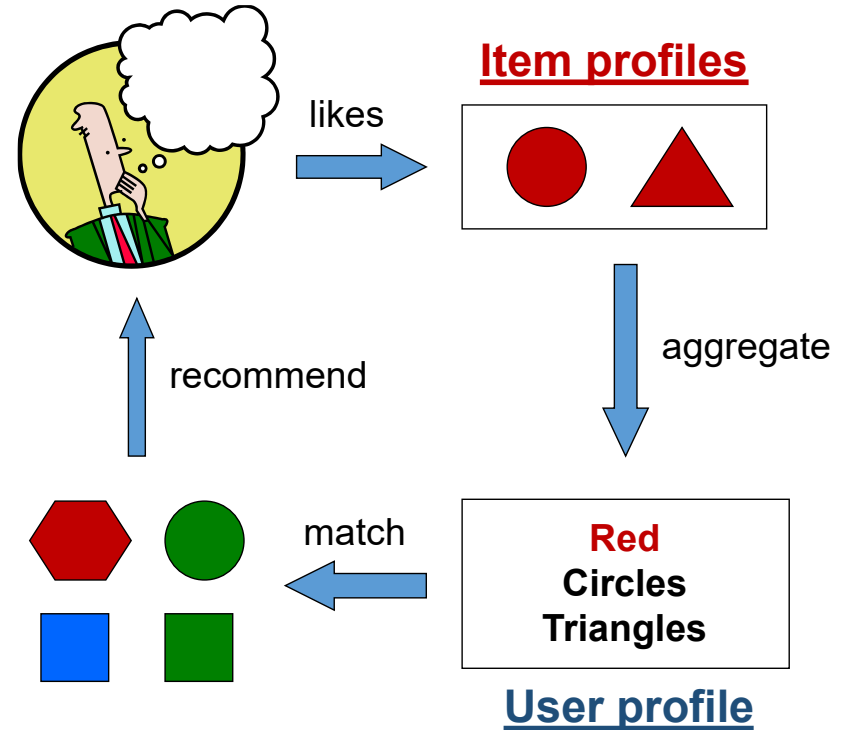
Main steps:

- **Item profiling** - use machine learning (e.g., **topic models**) or statistic methods (**TF-IDF**) to extract features of items based on the content
- **User profiling** - gather all the items of a user, and aggregate the features of those items
- Compute the “**similarity**” between the user and item profiles to compute a recommendation score
- Rank the items by the recommendation score and return top N items to the target user



Item profiles

- Each item has a profile
- The profile is often a vector, where each dimension represents a “**feature**”:
 - *menu* of a restaurant
 - *topics* or *important words* of a doc
- Features should contribute differently to an item
- TF-IDF: measure the importance of a feature



Recap: TF-IDF

- Term frequency – inverse document frequency (TF-IDF) is a well-known measure in information retrieval
- It models the importance of each word in the content of an item:

$$TF - IDF(w, d) = tf(w, d) \times idf(w)$$
$$tf(w, d) = f_{t,d}, \quad idf(w) = \log \frac{N}{|\{d \in D: w \in d\}|}$$

- Item profile p_v : each item is a “**document**” represented by a word vector, where each dimension is the tf-idf of the word if it appears in the text of the item, otherwise 0.

User profiles

- User profile q_u : aggregate the features of rated items
 - Unweighted: concatenate all rated item “documents” and compute the TF-IDF vector of user u
 - Weighted average: use rating to weight the item profiles
- Recommendation score $r(u, v)$: cosine similarity

$$r(u, v) = \cos(q_u, p_v) = \frac{q_u \cdot p_v}{\|q_u\| \cdot \|p_v\|}$$

Pros and cons – content-based approach

■ Pros

- 😊 Can recommend new items without any ratings
- 😊 Unpopular items can also be recommended
- 😊 Suitable for users with unique taste
- 😊 Easy to interpret the recommendation results

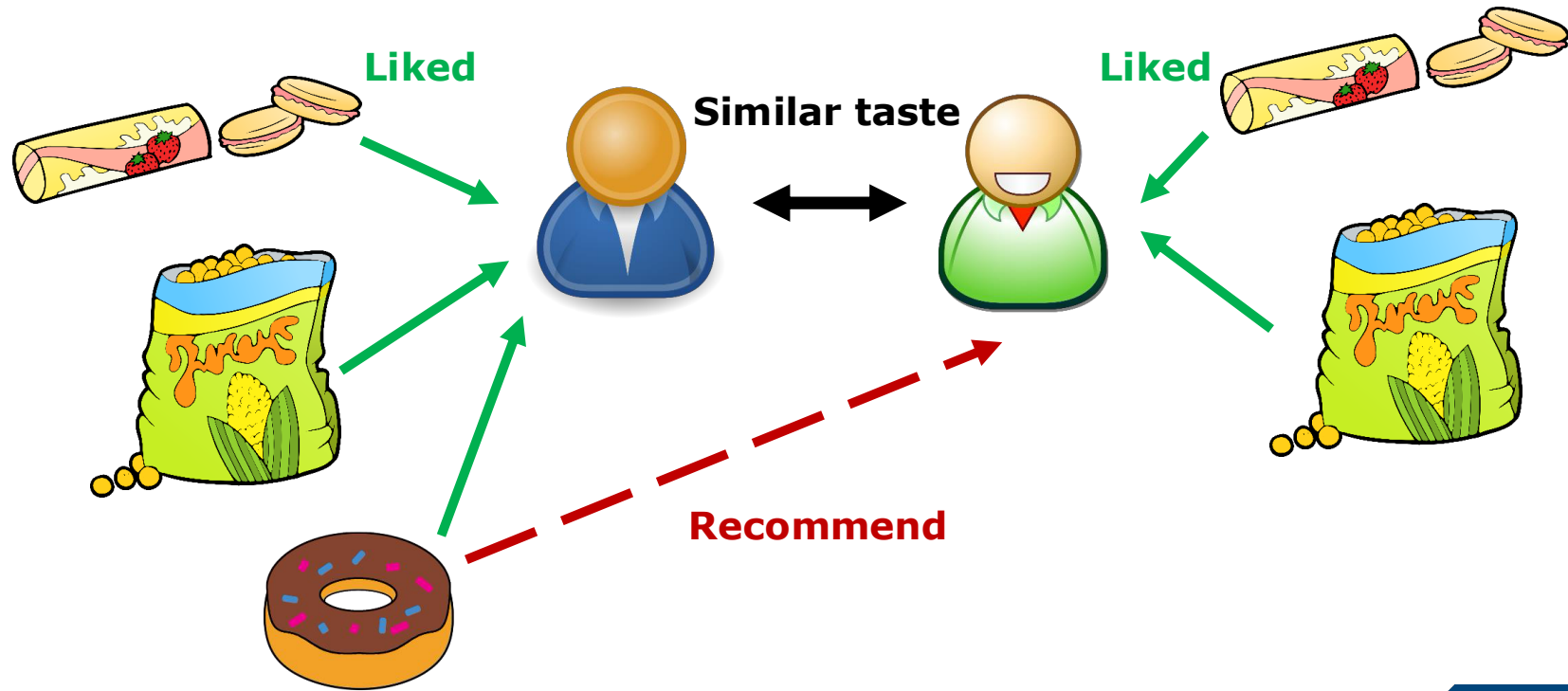
■ Cons

- 😞 Sensitive to feature selection and need to have the same set of features for all items
- 😞 Not work for new users
- 😞 Not suitable for users with multiple interests

Collaborative filtering

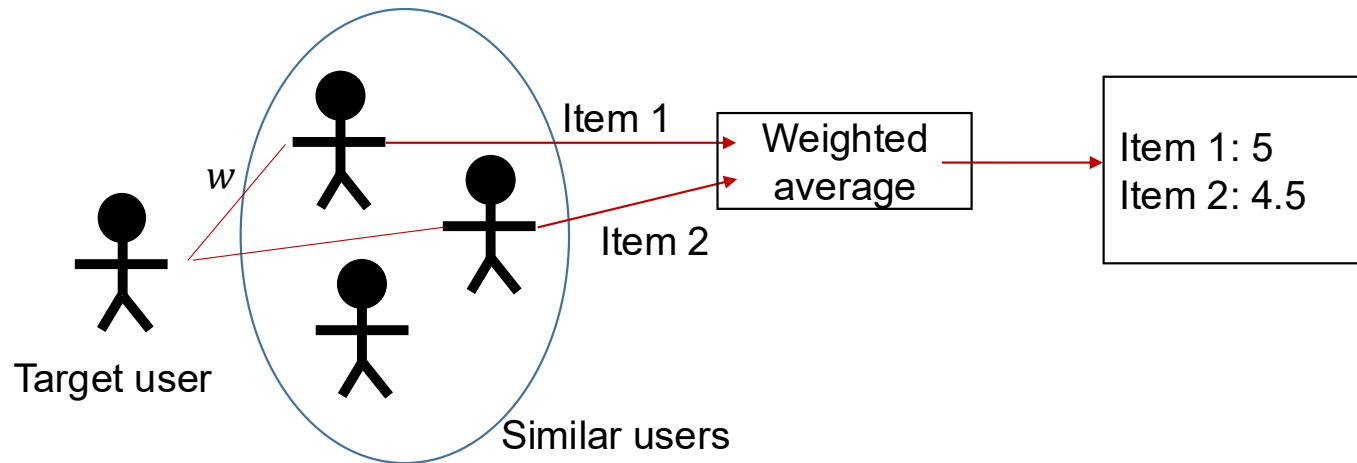
- Idea: learns the user preferences from the feedbacks
- Suppose we have the historical data of users
 - Explicit feedback – user ratings on items
 - Implicit feedback – clicks, scroll, time spent on a page
- Types of collaborative filtering
 - User-based
 - Item-based
 - Model-based

User-based collaborative filtering



User-based collaborative filtering

- Basic steps:
 - **Search similar users** – according to a **similarity function** over the historical behaviors on the items
 - **Prediction** – for each item adopted by similar users, compute estimate the rating that would be given by the user
 - **Ranking** – Pick the top N items and recommend them to the user



Similarity calculation

- Let $U = \{u_{1:n}\}$ be the set of users and $P = \{p_{1:m}\}$ be the set of items. r_{ij} be the rating from user u_i to item p_j
- Similarity between two users (u_i and u_k):
 - Cosine similarity

$$w_{ik} = \frac{\sum_j r_{ij} r_{kj}}{\sqrt{\sum_j r_{ij}^2} \cdot \sqrt{\sum_j r_{kj}^2}}$$

- **Problem:** Treats missing ratings as “negative”

Example

- Cosine similarity

$$w_{ik} = \frac{\sum_j r_{ij} r_{kj}}{\sqrt{\sum_j r_{ij}^2} \cdot \sqrt{\sum_j r_{kj}^2}}$$

$$\mathbf{r}_1 = (5, 0, 2, 0), \quad \mathbf{r}_2 = (3, 4, 0, 0)$$

$$\begin{aligned} w_{12} &= \frac{\mathbf{r}_1 \cdot \mathbf{r}_2}{\|\mathbf{r}_1\| \cdot \|\mathbf{r}_2\|} \\ &= \frac{5 \cdot 3 + 0 \cdot 4 + 2 \cdot 0 + 0 \cdot 0}{\sqrt{5^2 + 2^2} \sqrt{3^2 + 4^2}} \\ &= \frac{3}{\sqrt{29}} \end{aligned}$$

u_1	5	?	2	?
u_2	3	4	?	?
u_3	4	?	1	4
u_4	?	3	?	2
u_5	5	?	?	5
	p_1	p_2	p_3	p_4

Similarity calculation

- Let $U = \{u_{1:n}\}$ be the set of users and $P = \{p_{1:m}\}$ be the set of items.
 r_{ij} be the rating from user u_i to item p_j

- Similarity between two users (u_i and u_k):
 - Pearson correlation coefficient

$$w_{ik} = \frac{\sum_{j \in S_{ik}} (r_{ij} - \bar{r}_i)(r_{kj} - \bar{r}_k)}{\sqrt{\sum_{j \in S_{ik}} (r_{ij} - \bar{r}_i)^2} \cdot \sqrt{\sum_{j \in S_{ik}} (r_{kj} - \bar{r}_k)^2}}$$

- S_{ik} – the set of items rated by both users u_i and u_k
- $\bar{r}_i$ and $\bar{r}_k$ -- the average ratings of users u_i and u_k

Example

- Pearson correlation coefficient

$$w_{ik} = \frac{\sum_{j \in S_{ik}} (r_{ij} - \bar{r}_i)(r_{kj} - \bar{r}_k)}{\sqrt{\sum_{j \in S_{ik}} (r_{ij} - \bar{r}_i)^2} \cdot \sqrt{\sum_{j \in S_{ik}} (r_{kj} - \bar{r}_k)^2}}$$

Common item - 1st item

$$\begin{aligned} w_{12} &= \frac{(r_{11} - \bar{r}_1)(r_{21} - \bar{r}_2)}{\sqrt{(r_{11} - \bar{r}_1)^2} \cdot \sqrt{(r_{21} - \bar{r}_2)^2}} \\ &= \frac{(5 - 3.5)(3 - 3.5)}{|5 - 3.5||3 - 3.5|} \\ &= -1 \end{aligned}$$

u_1	5	?	2	?
u_2	3	4	?	?
u_3	4	?	1	4
u_4	?	3	?	2
u_5	5	?	?	5
	p_1	p_2	p_3	p_4

Prediction

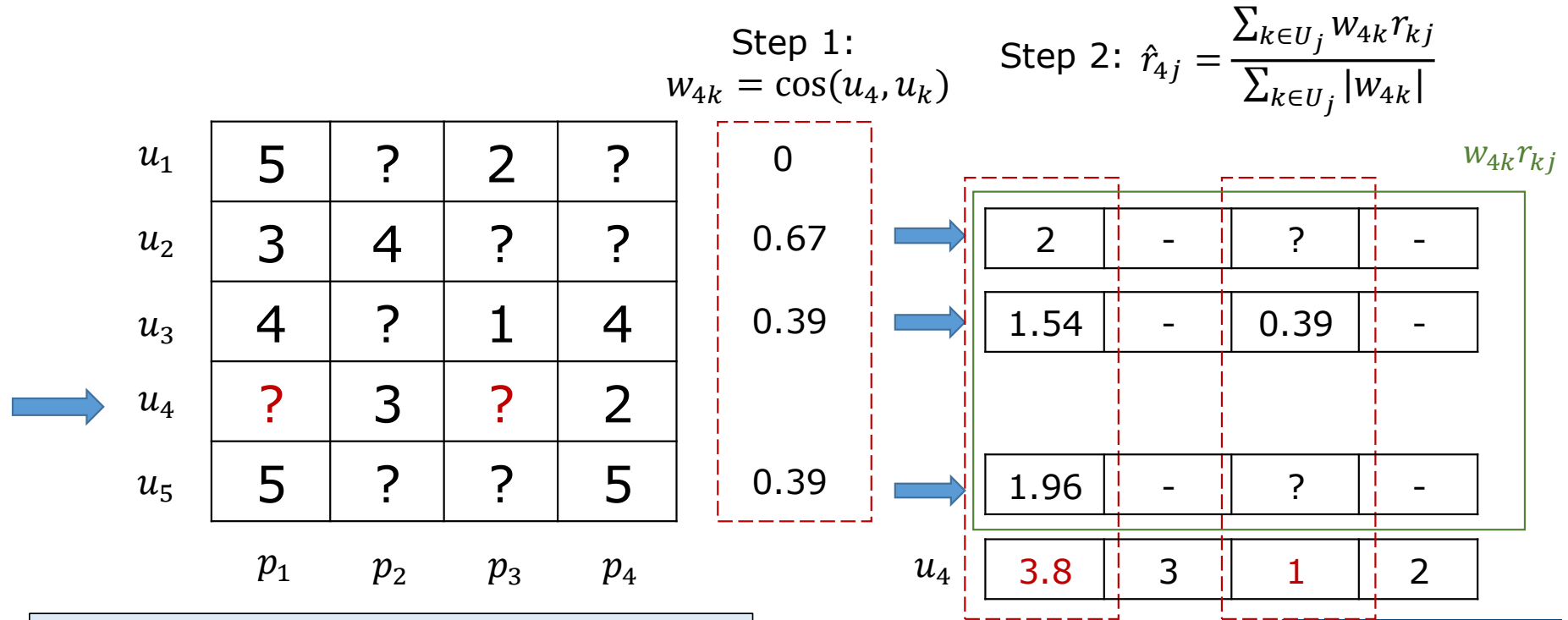
- Prediction – use the similarity to weight the ratings of items from other user u_k

$$\hat{r}_{ij} = \frac{\sum_{u_k \in U_j} w_{ik} r_{kj}}{\sum_{u_k \in U_j} |w_{ik}|}$$

U_j - set of most similar users who rated item p_j

User-based collaborative filtering

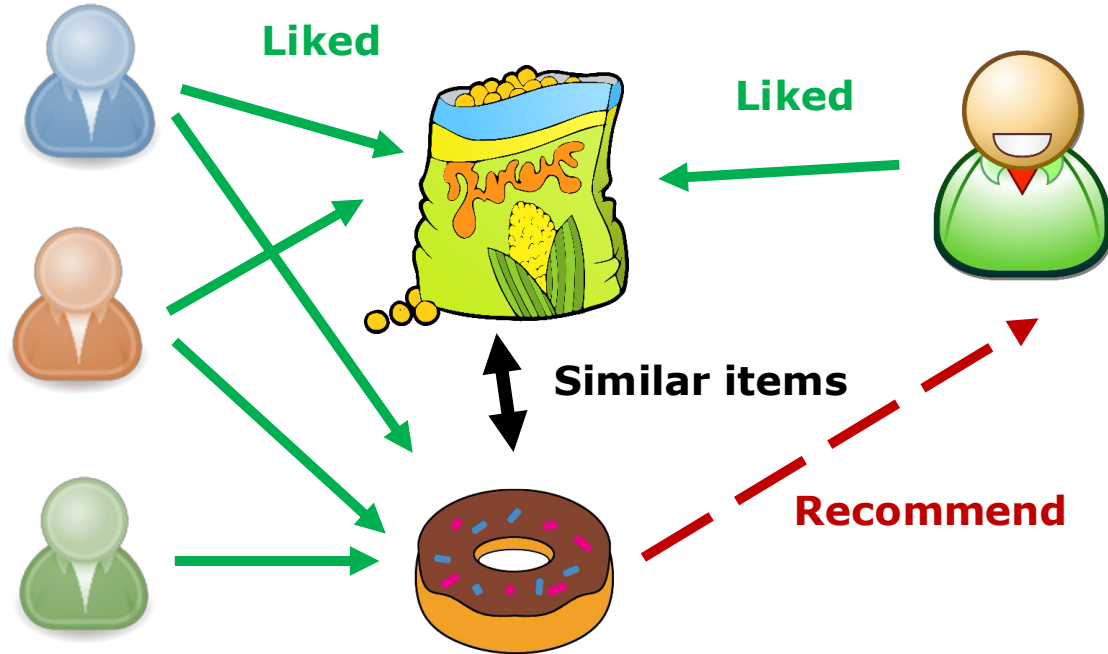
- User-based CF example



When predicting the rating, we can either consider
1) all the users or 2) only k most similar users

Item-based collaborative filtering

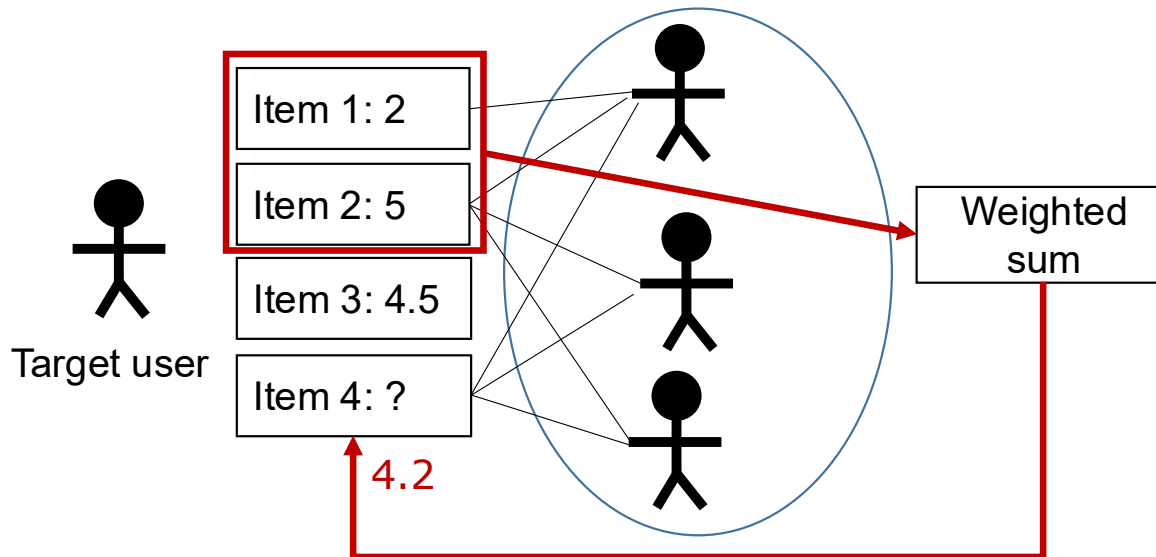
Another view:



Item-based collaborative filtering

- Basic steps:

- **Search similar items** – use the historical ratings from other users
- **Prediction** – Weighted **average on the most similar items** that have been rated by the target user.
- **Ranking** – Pick the top N items and recommend them to the user



Item-based collaborative filtering

- Let $U = \{u_{1:n}\}$ be the set of users and $P = \{p_{1:m}\}$ be the set of items. r_{ij} be the rating from user u_i to item p_j
- Finding similar items
 - Similarity – correlation between two items (p_j and p_l)

$$w_{jl} = \frac{\sum_{u_k \in U} (r_{kj} - \bar{r}_j)(r_{kl} - \bar{r}_l)}{\sqrt{\sum_{u_k \in U} (r_{kj} - \bar{r}_j)^2} \cdot \sqrt{\sum_{u_k \in U} (r_{kl} - \bar{r}_l)^2}} \quad \text{or} \quad w_{jl} = \frac{\sum_{u_k \in U} r_{kj} r_{kl}}{\sqrt{\sum_{u_k \in U} r_{kj}^2} \cdot \sqrt{\sum_{u_k \in U} r_{kl}^2}}$$

Pearson correlation Cosine similarity

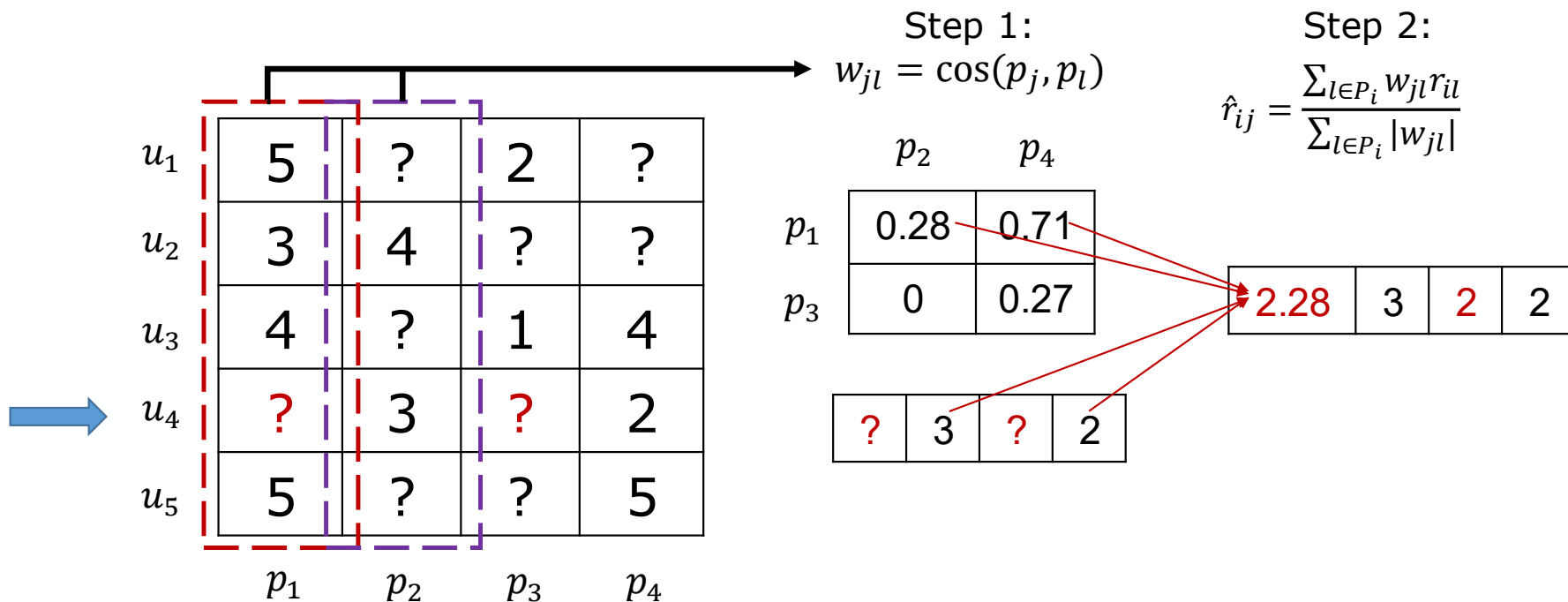
- Prediction – use the item similarity to weight the ratings from the other items rated by target user u_i

$$\hat{r}_{ij} = \frac{\sum_{l \in P_i} w_{jl} r_{il}}{\sum_{l \in P_i} |w_{jl}|}$$

P_i - set of most similar items rated by user u_i

Item-based collaborative filtering

- Item-based CF example



Pros and cons – collaborative filtering

■ Pros

- 😊 Don't need extra information from items
- 😊 Leverage the information from other users/items

■ Cons

- 😞 Cold-start problem
 - **New users** – user that has **no/limited ratings**, we need many rating records to construct accurate user profiles
 - **New items** – item that has **no/limited ratings** will not be recommended until it has been rated several times by users.
- 😞 Biased on popular items

Implementation details: global and local effects

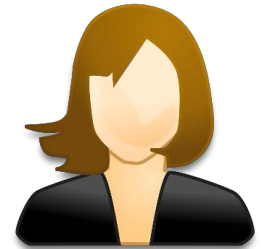
- Global effect:

- Mean pizza stores rating: **3.5 stars**
- *Pizzahut (item)* is **0.5** stars above avg.
- *Kate (user)* rates **0.2** stars below avg.
⇒ *Kate* may rate *Pizzahut* **3.8 stars**



- Local neighborhood (CF):

- Other users share similar interests with *Kate* had low ratings to *Pizzahut*
⇒ *Kate* may rate *Pizzahut* **3.6 stars**



Distinguish global and local terms

- **Global** effects

- b_g - (Global) Bias on all users and items, e.g., average of all ratings
 - $b_i^{(user)}$ - Bias on a user i over all items, e.g., $\bar{r}_i - b_g$
 - $b_j^{(item)}$ - Bias on an item j over all users, e.g., $\bar{r}_j - b_g$
- $$\Rightarrow b_{ij} = b_g + b_i^{(user)} + b_j^{(item)}$$

By distinguishing global and local terms, we often get better accuracy!

- **Local** neighborhood from CF

$$\hat{r}_{ij} = \frac{\sum_{u_k \in U_j} w_{ik} r_{kj}}{\sum_{u_k \in U_j} |w_{ik}|} \quad \longrightarrow \quad \hat{r}_{ij} = b_{ij} + \frac{\sum_{u_k \in U_j} w_{ik} (r_{kj} - b_{kj})}{\sum_{u_k \in U_j} |w_{ik}|}$$

Example of User-based CF

Rating matrix:

	p_1	p_2	p_3	p_4	p_5	p_6
u_1	?	5	?	4	?	3
u_2	?	2	4	2	4	?
u_3	3	?	4	?	?	2
u_4	?	4	1	?	1	?

Estimate the user u_4 's rating on item p_6

Use Pearson Correlation Coefficient
for user similarity computation

Step 1: Compute user similarity

$$w_{u_4, u_1} = \frac{(5 - 4)(4 - 2)}{\sqrt{(5 - 4)^2} \cdot \sqrt{(4 - 2)^2}} = 1 \quad w_{u_4, u_3} = \frac{(1 - 2)(4 - 3)}{\sqrt{(1 - 2)^2} \cdot \sqrt{(4 - 3)^2}} = -1$$

Step 2: Rating prediction

- Without the bias terms:

$$r(u_4, p_6) = \frac{1 \times 3 + (-1) \times 2}{|-1| + |1|} = 0.5$$

- With the bias terms:

$$b_g = \frac{39}{13} = 3$$

$$b_{u_1}^{(user)} = 4 - 3 = 1, \quad b_{u_3}^{(user)} = 3 - 3 = 0, \quad b_{u_4}^{(user)} = 2 - 3 = -1$$

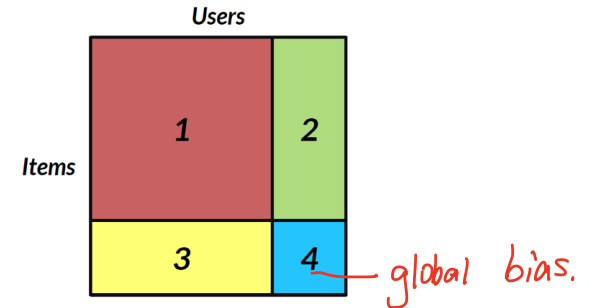
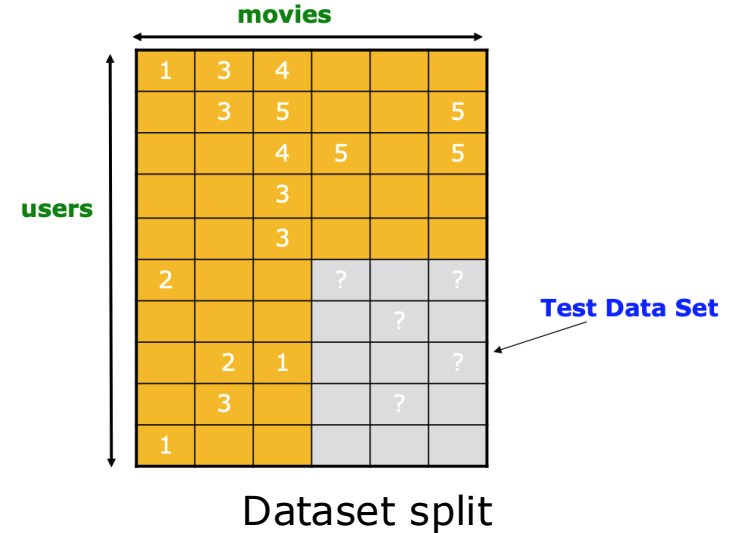
$$b_{p_6}^{(item)} = 2.5 - 3 = -0.5$$

$$b_{u_1, p_6} = 3 + 1 - 0.5 = 3.5, \quad b_{u_3, p_6} = 3 - 0.5 = 2.5, \quad b_{u_4, p_6} = 1.5$$

$$r(u_4, p_6) = 1.5 + \frac{1 \times (3 - 3.5) + (-1) \times (2 - 2.5)}{|1| + |-1|} = 1.5$$

Evaluation of RS

- How to get labeled data?
 - Hold-out some ratings/transactions
 - Random ratings
 - Latest ratings
- Cold-start scenarios
 1. **In-matrix prediction** - Interaction between existing items for existing users
 2. **User cold-start** - Existing items for new users
 3. **Item cold-start** - New items for existing users
 4. **User & Item cold-start** - New items for new users



Evaluation of RS

- **Rating Prediction Error**

- **Mean Square Error (MSE)**

$$MSE = \frac{1}{|R_{test}|} \sum_{i,j \in R_{test}} (r_{ij} - \hat{r}_{ij})^2$$

- **Root Mean Square Error (RMSE)**

$$RMSE = \sqrt{\frac{1}{|R_{test}|} \sum_{i,j \in R_{test}} (r_{ij} - \hat{r}_{ij})^2}$$

User	Item	Predicted $\hat{r}$	Groundtruth r
u_1	i_3	3.3	3.5
u_2	i_1	4	1
u_2	i_5	4.9	5
u_3	i_2	3.6	4

$$MSE = \frac{1}{4} (0.2^2 + 3^2 + 0.1^2 + 0.4^2) \approx 2.3$$

$$RMSE = \sqrt{MSE} \approx 1.5$$

Netflix
Competition

Rank	Team Name	Best Test Score	% Improvement	Best Submit Time
Grand Prize - RMSE = 0.8567 - Winning Team: BellKor's Pragmatic Chaos				
1	BellKor's Pragmatic Chaos	0.8567	10.06	2009-07-26 18:18:28
2	The Ensemble	0.8567	10.06	2009-07-26 18:38:22
3	Grand Prize Team	0.8582	9.90	2009-07-10 21:24:40

Evaluation of RS

- **Top-N recommendation** – return N items with the best scores
 - **Precision@N** and **Recall@N**

	Recommended	Not recommended
Preferred	True positive (tp)	False negative (fn)
Not preferred	False positive (fp)	True negative (tn)

$$Precision = \frac{\#tp}{\#tp + \#fp}$$

$$Recall = \frac{\#tp}{\#tp + \#fn}$$

Top-3
recommendations
for user u

i_3
i_5
i_7

Items that u
purchased/highly
rated in test set

i_2
i_5
i_9
i_{11}

For user u

$$Precision@3 = \frac{1}{1+2} = 0.33$$

$$Recall@3 = \frac{1}{1+3} = 0.25$$

The overall Precision@N and Recall@N are average on all users

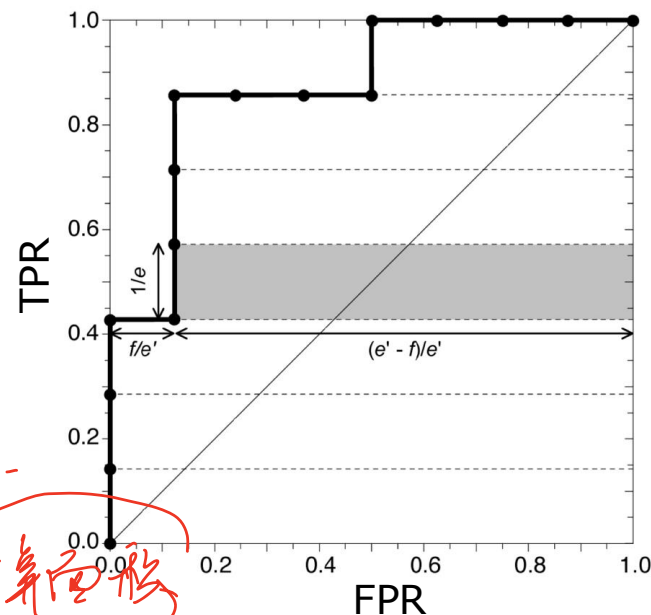
Evaluation of RS

- **Top-N recommendation** – return N items with the best scores
- **Receiver Operating Characteristic (ROC) curve**

- True positive ratio (TPR): $\frac{\#tp}{\#tp+\#fn} = \frac{\#tp}{|P_u^+|}$
- False positive ratio (FPR): $\frac{\#fp}{\#fp+\#tn} = \frac{\#fp}{|P_u^-|}$

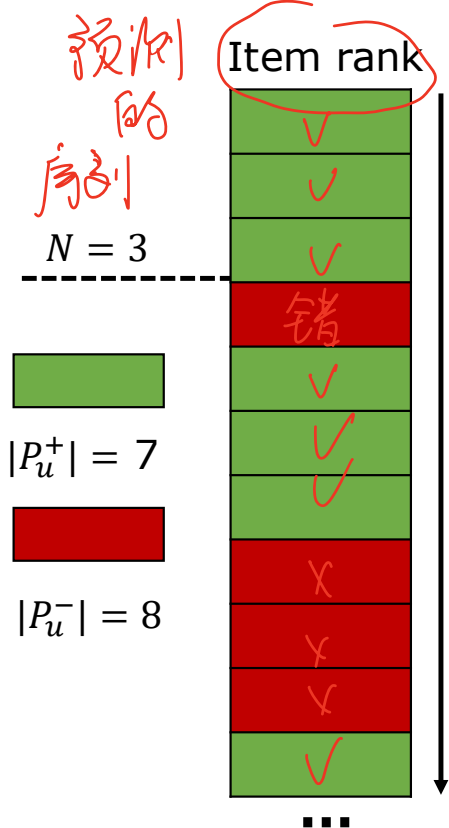
	Recommended	Not recommended
Preferred	tp	fn
Not preferred	fp	tn

Varying the size of recommendation set,
i.e., N to get the curve

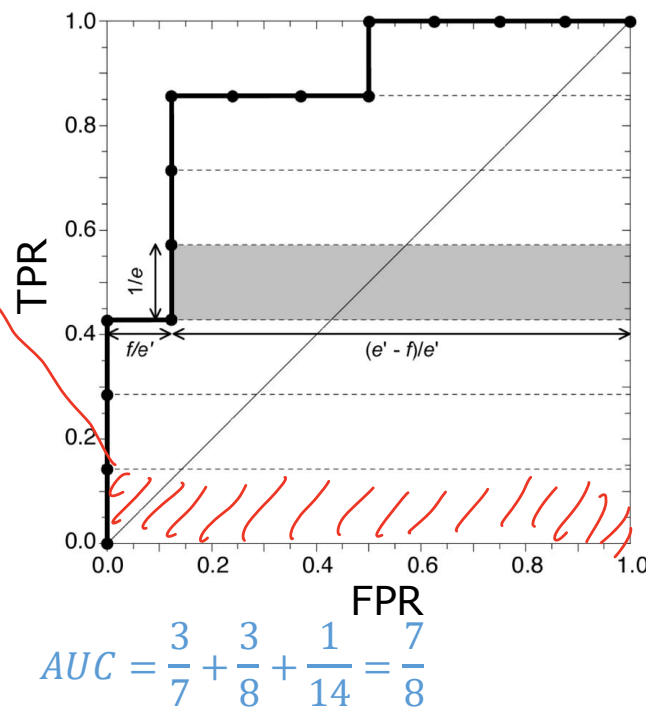


Evaluation of RS

- **Top-N recommendation** – return N items with the best scores
- **Area Under Curve (AUC)**



N	TPR	FPR	Area increment
1	1/7	0/8	$1/7 * 1 = 1/7$
2	2/7	0/8	$1/7 * 1 = 1/7$
3	3/7	0/8	$1/7 * 1 = 1/7$
4	3/7	1/8	0
5	4/7	1/8	$1/7 * (1 - 1/8) = 1/8$
6	5/7	1/8	$1/7 * (1 - 1/8) = 1/8$
7	6/7	1/8	$1/7 * (1 - 1/8) = 1/8$
8	6/7	2/8	0
9	6/7	3/8	0
10	6/7	4/8	0
11	7/7	4/8	$1/7 * (1 - 4/8) = 1/14$



Evaluation of RS

- **Top-N recommendation** – return N items with the best scores
 - **Area Under Curve (AUC)**

$$AUC(u) = \frac{\sum_{j \in P_u^-} \sum_{j' \in P_u^+} \mathbf{1}[f(j) < f(j')]}{|P_u^-| \cdot |P_u^+|}$$

- P_u^+ - the groundtruth set of items preferred by user
- P_u^- - the groundtruth set of items NOT preferred by user

